TALLER DE PROGRAMACIÓN / CLASE #04

Guía Teórica - Docentes

1. Funciones con parámetros
2. Ejemplos prácticos
99. Ejercitación propuesta

1. Funciones con parámetros

Un parámetro es una variable declarada entre los paréntesis de la función, que actúa como "receptor" de un valor entregado al llamarla. El valor real que se envía en la llamada se denomina argumento.

```
def saludar(nombre):   # 'nombre' es el parámetro
    print(f"Hola, {nombre}")

saludar("Ana")   # "Ana" es el argumento
```

Las ideas fundamentales

- Parámetro vs. argumento — el parámetro es el nombre en la definición; el argumento es el valor real enviado al llamar la función.
- Múltiples parámetros — una función puede recibir varios, separados por coma: def sumar(a, b):
- Argumentos posicionales — se asignan en el mismo orden en que aparecen los parámetros.
- Argumentos por nombre (keyword arguments) — se pueden pasar indicando el nombre del parámetro, sin importar el orden: saludar(nombre="Ana")
- Valores por defecto — un parámetro puede tener un valor predefinido que se usa si no se pasa el argumento: def saludar(nombre="invitado"):
- Alcance (scope) — los parámetros son variables locales: solo existen dentro de la función.

Conviene indagar primero funciones con un solo parámetro, y recién después combinar varios, argumentos por nombre y valores por defecto. Insistir en que el orden de los argumentos posicionales importa, pero el de los argumentos por nombre no.

2. Ejemplos prácticos

Ejemplo 1 — Un parámetro simple

```
def es_par(numero):
    if numero % 2 == 0:
        print(f"{numero} es par")
    else:
        print(f"{numero} es impar")
```

```
es_par(7)
```

Ejemplo 2 – Varios parámetros posicionales

```
def calcular_rectangulo(base, altura):
    area = base * altura
    print(f"El área es {area}")

calcular_rectangulo(5, 3)
```

Ejemplo 3 – Argumentos por nombre

```
def presentar(nombre, edad, ciudad):
    print(f"{nombre} tiene {edad} años y vive en {ciudad}")

presentar(edad=20, nombre="Lucía", ciudad="Rosario")
```

Ejemplo 4 – Valores por defecto

```
def calcular_precio(precio, descuento = 0):
    total = precio - (precio * descuento / 100)
    print(f"Precio final: ${total}")

calcular_precio(1000)          # usa descuento=0
calcular_precio(1000, 15)      # usa descuento=15
```

Ejemplo 5 – Combinando posicionales, nombre y valores por defecto

```
def crear_perfil(usuario, edad, pais = "Argentina"):
    print(f"Usuario: {usuario} | Edad: {edad} | País: {pais}")

crear_perfil("martu22", 19)
crear_perfil(usuario = "db_king", edad = 25, pais = "México")
```

99. Ejercitación propuesta

EJ01) Definir una función **mostrar_saludo(nombre)** que imprima "Bienvenido/a, < nombre >".

EJ02) Definir una función **es_mayor_de_edad(edad)** que indica si una persona es mayor o menor de edad. (devuelva el booleano correspondiente)

EJ03) Definir una función **sumar_dos(a, b)** que reciba dos números y describa la suma.

EJ04) Definir una función **calcular_promedio(nota1, nota2, nota3)** que reciba tres notas y describa el promedio.

EJ05) Definir una función **calcular_imc(peso, altura)** que calcule el Índice de Masa Corporal (peso / altura**2) y muestre un mensaje según el resultado (bajo peso, normal, sobrepeso).

EJ06) Definir una función **convertir_moneda(monto_dolares, tasa = 1400)** que convierta dólares a pesos, usando tasa como valor por defecto pero permitiendo pasarlo por nombre.

EJ07) Definir una función **describir_mascota(nombre, especie = "perro")** que use un valor por defecto para la especie, y probarla con y sin ese argumento.

EJ08) Definir una función **generar_factura(producto, precio, cantidad, descuento = 0)** que describa el producto y el precio total con descuento aplicado y formateado con dos decimales.

EJ09) Definir una función **validar_contraseña(contraseña, longitud_min = 8)** que verifique si la contraseña cumple, propuesta por parámetro, con la longitud mínima, también declarado por parámetro; y contiene al menos un número (usar un bucle o comprensión). Como mensaje de salida, puede utilizar “contraseña válida” / “contraseña invalida”

EJ10) Definir una función **calcular_envio(peso, distancia, es_urgente = False)** que calcule el costo de un envío combinando varias reglas: costo base según peso (en kg), recargo por distancia (en km), y recargo adicional del 50% si es_urgente es True. Debe poder llamarse con argumentos posicionales y por nombre indistintamente. Por kilogramo se debe multiplicar por 1000 y la distancia por un factor de 7, 2.

Ejercicio Integrador:

Estás armando el sistema de pedidos de una heladería. Cada bocha de helado cuesta $700, salvo que el sabor elegido sea "premium" (dulce de leche, pistacho), en cuyo caso cada bocha sale $900. Además, hay toppings opcionales (chips, crema, salsa) que suman $200 fijos al pedido si se agregan, o nada si no se pide ninguno.
Armá una función calcular_precio(bochas, sabor="común", con_topping=False) que reciba la cantidad de bochas, el tipo de sabor (con valor por defecto "común") y si lleva topping o no (con valor por defecto False), y devuelva el precio total del pedido, aplicando la lógica de arriba con if/elif.
Después, armá una función aplicar_descuento(precio, descuento=0) que reciba un precio y un porcentaje de descuento (0 por defecto) y devuelva el precio final ya descontado.
Tenés una lista de pedidos del día, donde cada uno es una lista con el nombre del cliente, la cantidad de bochas, el sabor y si lleva topping — por ejemplo ["Juan", 2, "premium", True]. Recorre la lista con un for y, para cada pedido, llamá a calcular_precio() pasando los datos correspondientes. Si el precio de ese pedido supera los $2000, aplicale un 10% de descuento llamando a aplicar_descuento(); si no llega a esa cifra, no le apliques descuento (podés pasar descuento=0 explícitamente o directamente no llamarla).
Muestre en pantalla, para cada cliente, cuánto pagó finalmente. Al terminar de recorrer todos los pedidos, muestre el total recaudado por la heladería en el día.